

Input and Output

Input and Output (in MIMD systems)

Input and output on MIMD systems can be fairly *complicated*

As of now,

- most of our input and output systems are designed around using the standard `C` library

`printf` , `scanf` , etc

If you had multiple cores, and each core executed a `printf` , where do you want the output to go?

Input and Output (in MIMD systems)

Parallel and distributed systems are powerful but usually struggle with input and output

The funnel problem

In a distributed system, we might have thousands of distinct cores running a program

But they usually share a **single** physical connection to the terminal where you typed the run command

Limitations of the library

`printf`, `scanf`, etc., were designed for single core systems and assume a single stream of executions.

I/O is serial by nature, parallelizing it usually creates bottlenecks

Non-determinism

Any input and output from multiple threads can be **interleaved** in unpredictable ways

For example, if two threads execute

```
1 printf("Hello");
```

and

```
1 printf("World");
```

the output could be

```
1 Hello
2 World
```

or

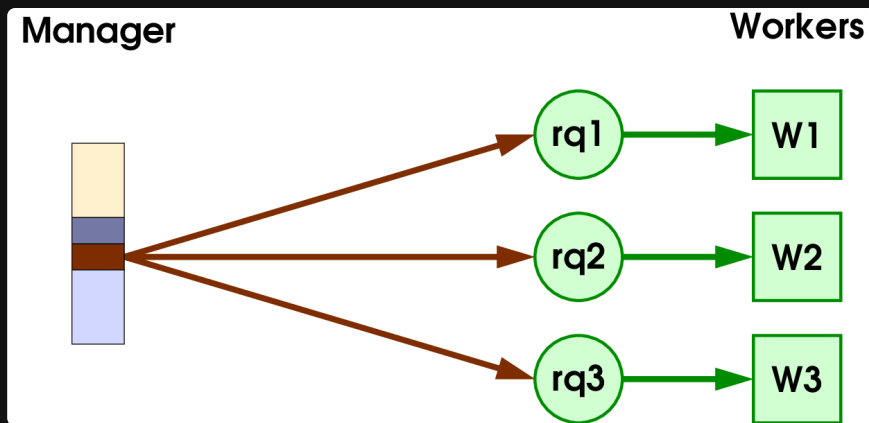
```
1 World
2 Hello
```

Conventions for Input and Output

To address some of these input and output issues

We simply make some assumption and follow certain **conventions**

The "Manager" Pattern



Simply have only `rank 0` talk to the user and read from the keyboard

1. `rank 0` reads data
2. `rank 0` distributes data to other ranks
3. all ranks do their work
4. `rank 0` collects results from other ranks
5. `rank 0` outputs results

on File access

If 1,000 different processes try to write to the `results.txt` file,

The system might

- lock up,
- corrupt the file,
- crash the program,
- or simply not write the expected output

The solutions to this are either

1. Simply **serialize** I/O, where only one process can read or write at a time
2. Use a specialized parallel I/O library, such as `MPI-IO` or `HDF5`

on Debugging

When debugging regular programs, we often use `printf` statements to print out variable values

In parallel programs, this method also *works*, but requires **one** important consideration

Always print the rank of the process or thread that is printing the output

And also make sure each core can write to `stdout` or `stderr`

GPU considerations

GPU architecture is *fundamentally different* from CPU architecture

The **host** cpu, the one setting up the GPU code, is separate from the GPU itself

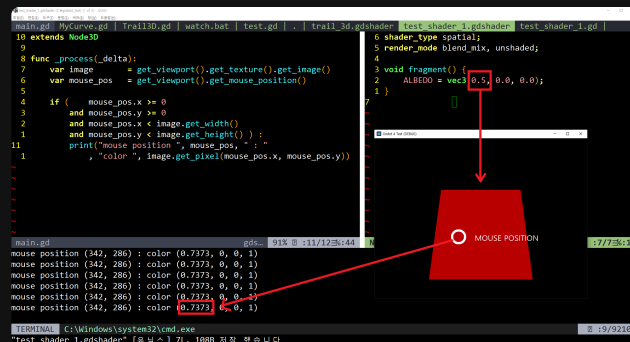
And so all input and output is done through the **host**

With one exception:

debugging

In this scenario, we'll direct the output of each core to `stdout` or `stderr`, which will be flushed to the CPU at some point

The order of the `output` is random



Though usually, debugging GPU code involves creating custom solutions

Such as making interactive visualizations, writing data to files, or using specialized debugging tools